

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

IoT Security

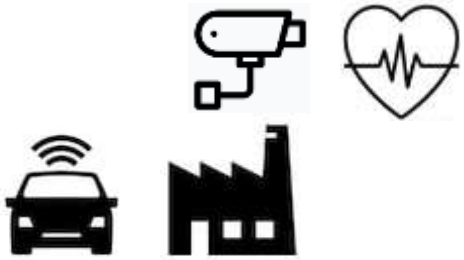
Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Internet of Things, 2025/2026

Security in IoT

- Very important (probably more than for standard Internet) due to the type and value of:
  - **information that is exchanged/stored**
  - **services that are implemented**
    - in terms of damage and/or possible downtime



- Notable IoT scenarios:
  - **Internet of Medical Things IoMT**
    - connected devices in healthcare
      - e.g. monitoring devices, robotic surgery
  - **Smart Cities**
  - **Industrial IoT (IIoT)**
    - Industry IT and OT systems are becoming more integrated an interconnected to external systems

Main issues in IoT security

- In the past most attacks concerned traditional computer systems and computer networks
  - **most of the security mechanisms and protocols has been developed for standard Internet-connected systems**
- IoT brought new security challenges
  - **presence of different highly heterogenous devices (sensors, actuators, etc.)**
  - **devices may suffer from their own vulnerabilities, especially in case of very constrained devices**
    - limited computational power, memory capabilities, communication resources, lack of a user interface
  - **use of different communication protocols, sometimes proprietary protocols designed to be used in protected networks**
  - **applications may be not well tested**
  - **difficulty to apply patches and updates impacts software life cycle**
  - **presence of different attack surfaces and vectors**
    - also for isolated networks

Attack domains

Different attack domains:

- Physical and Hardware
  - **related to the hardware used within the different systems, and may consist in the replacement, destruction, modification or exploitation of hardware components (chips, circuit boards, device ports, or other)**
    - no or limited tamper resistance policies and methodologies
- Communications
  - **exploitation of possible vulnerabilities of communication technologies (both wired and wireless) and protocols at different layers (data-link, network/routing, transport, or application)**
- Software
  - **attacks that try to exploit possible application weaknesses in design or in the implementation**

Different attack domains

- Hardware
  - **attacks related to the physical access of a system, exploiting possible weaknesses in its physical security, and related to the hardware used by different systems**
    - some devices may operate in an unattended fashion
    - may consist in the replacement, destruction, modification or exploitation of hardware components (chips, circuit boards, device ports, or other), e.g.
      - extraction of security parameters from physically unprotected devices
        - » if a group key is used and compromised this way, the whole network may be compromised as well
      - firmware replacement attack
        - » e.g. when a thing is in operation or maintenance phase, its firmware or software may be updated/replaced to allow for new functionality or new features
      - cloning of smart things by untrusted manufacturers and/or malicious substitution of things during the installation
        - » e.g. with lower-quality things, untrusted manufacturer may change functionality of the cloned thing, resulting in degraded functionality

Different attack domains (cont.)

- Communications
  - **exploitation of possible vulnerabilities of communication technologies (both wired and wireless) and protocols at different layers (data-link, network/routing, transport, or application)**
  - **these attacks may lead to data interception (steeling information), modification, communication hijacking and/or deny of service (DoS), e.g.**
    - eavesdropping attacks
      - if the communication channel (often a wireless medium) is not adequately protected
        - » keying materials or security parameters/protocols not correctly set-up
      - an attacker can also gather and track thing's information, deducing behavioral patterns
    - modification and Man-in-the-middle attacks
      - particularly during key establishment and/or device authentication procedure
        - » device authentication/authorization may be not automated and need support of a human decision process, since things usually do not have a priori knowledge about each other

Different attack domains (cont.)

- Communications (cont.)
  - routing attacks
    - routing information in IoT can be spoofed, altered, or replayed, in order to create routing loops, attract/repel network traffic, extend/ shorten source routes, etc.
    - relevant routing attacks may include:
      - » Sinkhole attack (or blackhole attack): an attacker declares himself to have a high-quality route/path to the base station, thus allowing him to do anything to all packets passing through it
      - » Selective forwarding: an attacker may selectively forward pkts or simply drop a packet
      - » Sybil attack: an attacker presents multiple identities to other things

Different attack domains (cont.)

- Communications (cont.)
  - Denial-of-Service (DoS) attacks
    - typically, things have tight memory and limited computation, they are thus vulnerable to resource exhaustion attack
      - » attackers can continuously send requests to be processed by specific things so as to deplete their resources
      - » this is especially dangerous in the IoTs since an attacker might be located in the backend and target resource-constrained devices in an LLN (Low power and Lossy network)
    - DoS attack can be launched by physically jamming the communication channel
      - » thus breaking down the T2T communication channel
    - network availability can also be disrupted by flooding the network with a large number of packets

Different attack domains (cont.)

- Software
  - **attacks that try to exploit possible application weaknesses in design or in the implementation**
    - due to very low maturity of the defined protocols and architecture
      - new software implementations poorly tested
        - » firmware are often released with known vulnerabilities (including backdoors, root users as prime access points) and lack of data encryption usage
      - weak configurations
        - » Various IoT devices have unnecessarily open ports while running vulnerable services, permitting an attacker to connect and exploit a plethora of vulnerabilities
        - » IoT operating systems and embedded firmware/software should be patched appropriately to continuously minimize attack vectors
      - malware

OWASP IoT Top 10 Vulnerabilities

1) Weak Guessable, or Hardcoded Passwords
   - **easy to bruteforce**
   - **publicly available**
     - e.g. https://defpass.com
   - **unchangeable credentials**

2) Insecure Network Services
   - **unneeded or insecure network services running on the device**
     - may be connected to the Internet
     - may lead to unauthorized access to the device

3) Insecure Ecosystem Interfaces
   - **insecure interfaces in the ecosystem outside the device: Web, backend API, cloud, or mobile interfaces**
   - **Common issues:**
     - lack of or weak authentication, authorization, encryption, lack of input and output filtering

Source: Open Web Application Security Project (OWASP), 2018

OWASP IoT Top 10 Vulnerabilities (cont.)

4) Lack of Secure Update Mechanism

- **lack of ability to securely update the device**
  - lack of firmware validation on device and/or secure delivery (un-encrypted in transit)
  - lack of anti-rollback mechanisms or notification of security changes due to updates

5) Use of Insecure or Outdated Components

- **use of deprecated or insecure software components/libraries that could allow the device to be compromised**
  - including third-party software libraries or third-party hardware components from a compromised supply chain

6) Insufficient Privacy Protection

- **user's personal information stored on the device or in the ecosystem that is used insecurely, improperly, or without permission**

OWASP IoT Top 10 Vulnerabilities (cont.)

7) Insecure Data Transfer and Storage

- **lack of encryption or access control of sensitive data anywhere within the ecosystem, including at rest, in transit, or during processing**
  - e.g. most of the times, no TLS or other security communication protocols

8) Lack of Device Management

- **lack of security support on devices deployed in production, including asset management, update management, secure decommissioning, systems monitoring, and response capabilities**

9) Insecure Default Settings

- **devices or systems shipped with insecure default settings**

10) Lack of Physical Hardening

- **allowing malicious users to gain sensitive information that can help in a future remote attack or take local control of the device**

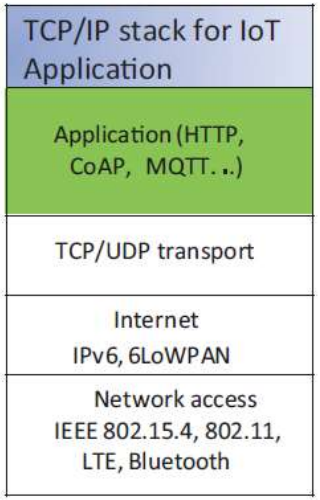
Possible countermeasures

- Countermeasures against physical threats:
  - **security policies**
  - **when possible, protect smart objects and/or apply physical hardening**
  - **safe supplying and installation measures**
    - avoiding untrusted manufacturers and installers
- Countermeasures against communication threats:
  - **security policies on which services and protocols should be used**
  - **secure communication protocols and cryptographic algorithms**
    - to enforce peer authentication/authorization, data protection (authentication/integrity, confidentiality)
    - using proper cryptographic tools
      - ciphers, hash functions, solid key management infrastructure
    - more complicated in IoT scenarios than in standard Internet
- Countermeasures against software threats:
  - **use of well-tested and updated software**
  - **security policies in all software life-cycle**
    - including software update mechanisms

IoT protocols

- A number of protocols have been developed for the IoT
  - **6LowPAN, is located between the network layer and the link layer of the OSI model**
  - **while others are at application level, such as**
    - Message Queuing Telemetry Transport (MQTT)
    - Constrained Application Protocol (CoAP)
  - **other protocols and systens spread on different layers like, e.g.**
    - LoRaWAN

- Security mechanisms and protocols may apply on different layers



Selecting security protocols

- Data authentication, integrity, and confidentiality might be provided at PH and/or at DL layer (e.g., in IEEE 802.15.4 systems), however,
  - **only hop-by-hop security**
    - end-to-end security can be guaranteed only with a high level of trust on intermediate nodes
    - difficult to guarantee due to the dynamic nature of L2 technologies and routing protocols
- For such a reason, security mechanisms at network, transport, or application level should be considered instead of (or in addition to) PH and DL level security
- Standard application protocol like MQTT, CoAP also define their security mechanisms
  - **ofthen using application level authentication and transport layer security**
    - TLS and DTLS

Selecting security protocols (cont.)

- Some important aspects:
  - **use standard secure protocol, with standard cryptographic algorithms**
    - like IPSec, TLS/DTLS
  - **possibly avoid security service duplication**
    - it has impact on the computation and transmission performance
  - **interoperability**
    - many of the Internet security protocols are independent by the used security algorithms
      - algorithms and parameters sometime are negotiated between the communication parties
    - for full interoperability, and to reduce the number of implemented cipher suites, it is important to define a set of mandatory algorithms (targeted for constrained environments) that all objects must implement for minimal support
      - however it is not easy to agree on a set of common protocols and algorithms that can be supported by all devices

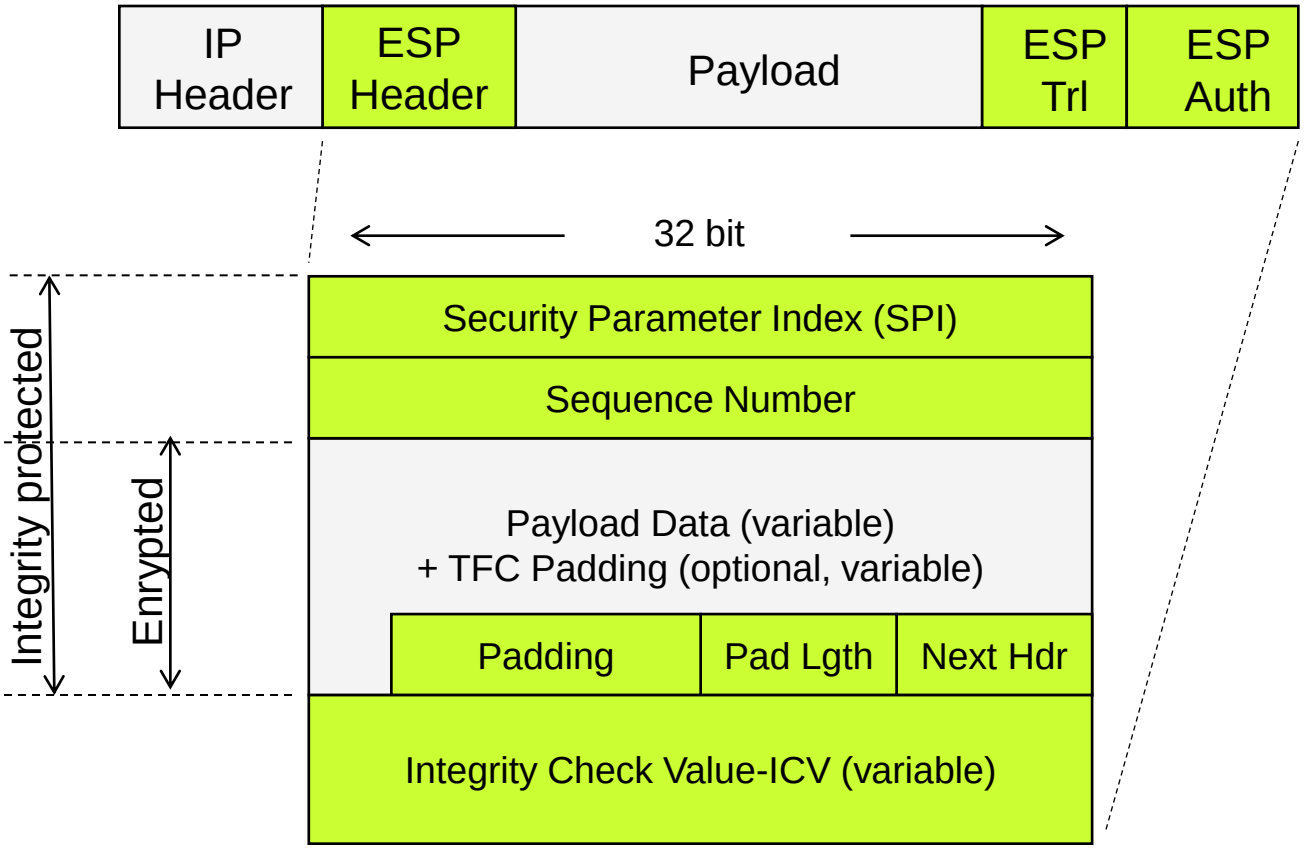
IPSec

- At network layer, an IoT node can secure data exchange in a standard way by using the Internet Protocol Security (IPSec)
- IPSec (RFC 4301) is the Security Architecture for the IP protocol
  - **defined for both IPv4 and IPv6**
    - it was originally developed for IPv6 but found widespread deployment, first, as an extension in IPv4, into which it was back-engineered
    - it was an integral part of the base IPv6 protocol suite, but has been made optional
- IPSec can provide:
  - **confidentiality**
  - **integrity**
  - **data-origin authentication**
  - **protection against replay attacks**

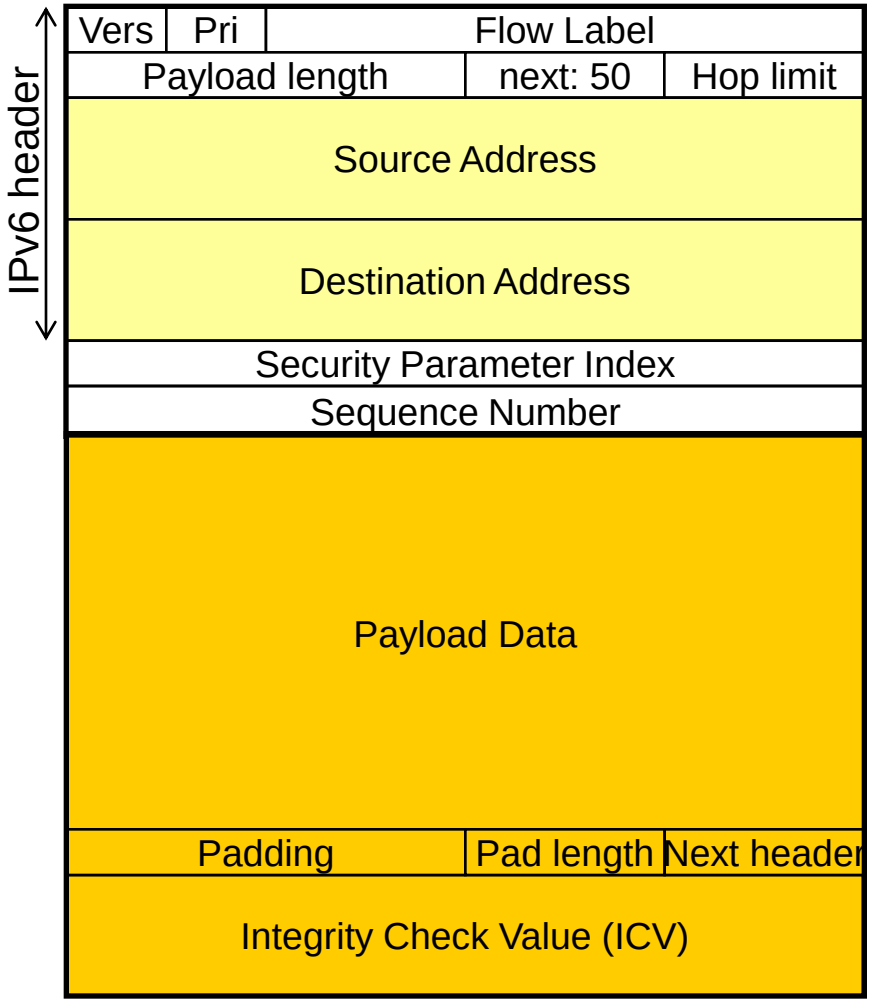
IPSec protocols

- Such security services are implemented by two IPSec security protocols:
  - **Authentication Header (AH)**
    - provides data-origin authentication, integrity, and optionally anti-replay
  - **Encapsulated Security Payload (ESP)**
    - provides confidentiality, data-origin authentication, integrity, and anti-replay capabilities
    - usually preferred since it is all-in-one solution
      - respect to a combination of AH+ESP
- In order to minimize the code, only ESP is implemented and used

IPSec ESP packet format



IPSec ESP packet format (cont.)



- Security Parameter Index (SPI)
- Sequence number
- Payload Data
  - **may including the cipher Initialization Vector**
  - **IPsec implementation can add also Traffic Flow Confidentiality (TFC) padding**
- Padding and Pad length
  - **If the encryption algorithm requires the plaintext to be a multiple of some number of bytes, e.g., the block size of a block cipher**
  - **or to ensure that the resulting ciphertext terminates on a 4-byte boundary**
- Next header
  - **payload type**
- Integrity Check Value (ICV)

IPSec (cont.)

- IPSec ESP defines only the way IPSec control information are encapsulated
  - **the effective security algorithms are specified separately algorithms can be selected amongst a set of available cipher suites**
  - **this makes IPSec usable also in the presence of very resource-constrained devices, if a proper algorithm that guarantees both usability and sufficient security level is selected**
- The keying material and the selected cryptographic algorithms used by IPSec for securing a communication (Security Association) can be:
  - **pre-configured (specifying a priori a pre-shared key, hash function and encryption algorithm) or**
  - **dynamically negotiated by the IKE protocol**
- Unfortunately the IKE protocol uses asymmetric cryptography, which may be computationally too heavy for very small devices
  - **proper IKE extensions may be considered using lighter algorithms**

Transport-level security

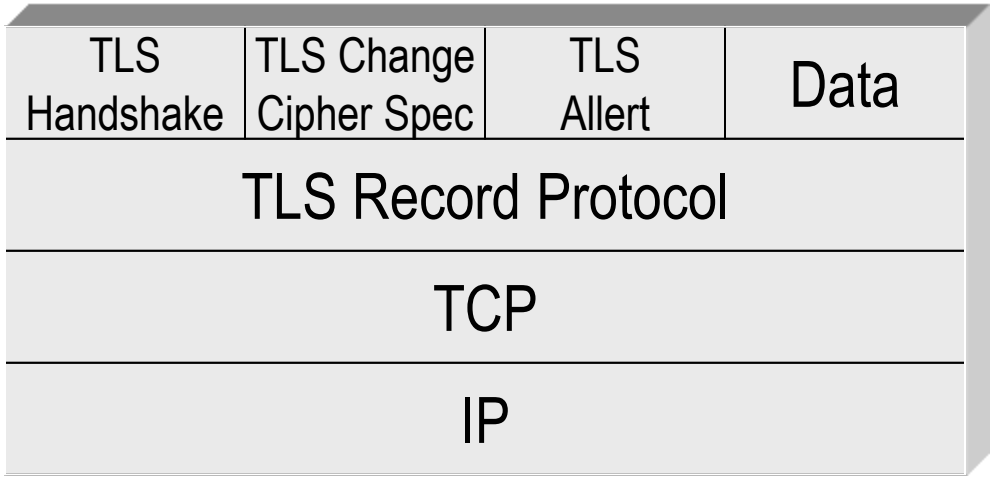
- Similar to the standard Internet, data exchange between applications can be secured at transport layer through
  - **Transport Layer Security (TLS)**
    - widest used secure protocol, running on top of the TCP providing to the application layer the same connection and stream-oriented interface of TCP
  - **Datagram Transport Layer Security (DTLS)**
    - introduced in order to provide a security service similar to TLS on top of UDP
    - can be used to secure datagram traffic for client/server applications

Transport Layer Security (TLS)

- Transport Layer Security (TLS) protocol (version 1.3, 2018)
  - **widest used secure protocol, running on top of the TCP**
  - **same connection and stream-oriented interface of TCP**
- Provides privacy and data integrity between two communicating client/server applications
  - **peer's identity authentication**
    - using asymmetric cryptography (e.g., RSA, DSA, etc.)
  - **negotiation of encryption algorithm and key exchange**
    - protected against eavesdropping, modifying and replay attacks
  - **confidentiality**
    - symmetric cryptography is used
  - **data authentication and integrity**
    - message transport includes a message integrity check using a keyed MAC (e.g. based on SHA-1 hash)
- Uses X.509 certificates

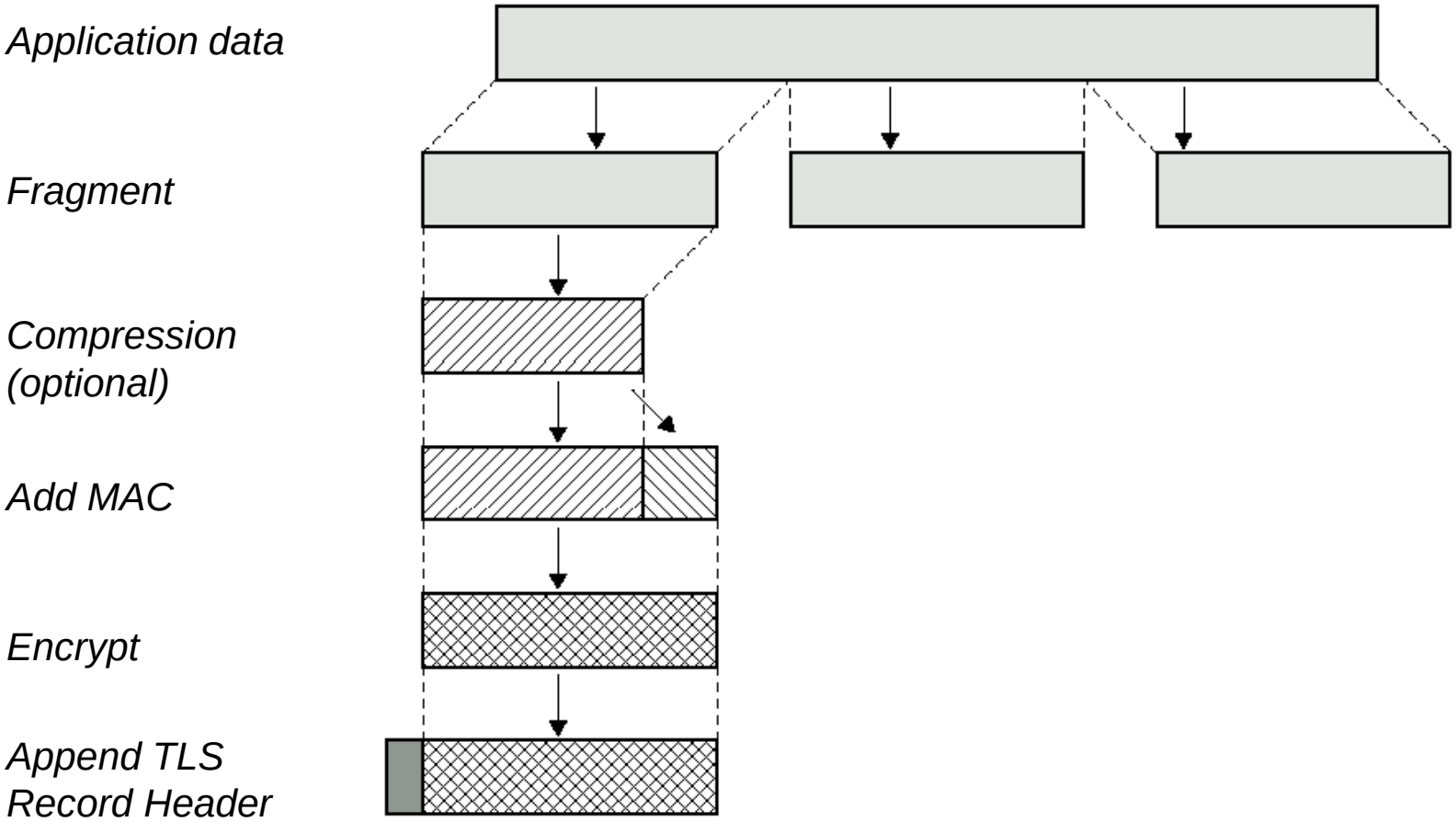
TLS protocol stack

- The protocol is composed of two layers:
  - **TLS Handshake Protocol**
  - **TLS Record Protocol**



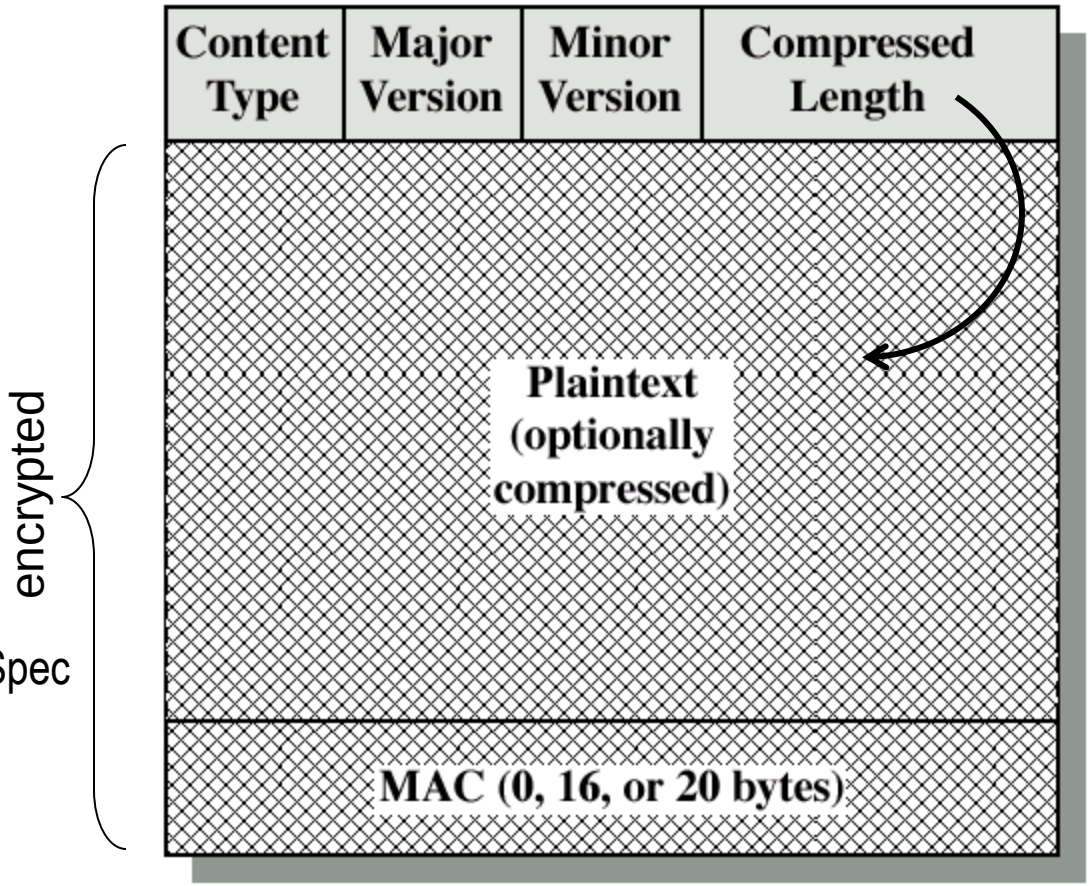
TLS Record Protocol (cont.)

- TLS-RP operation:



TLS Record Protocol (cont.)

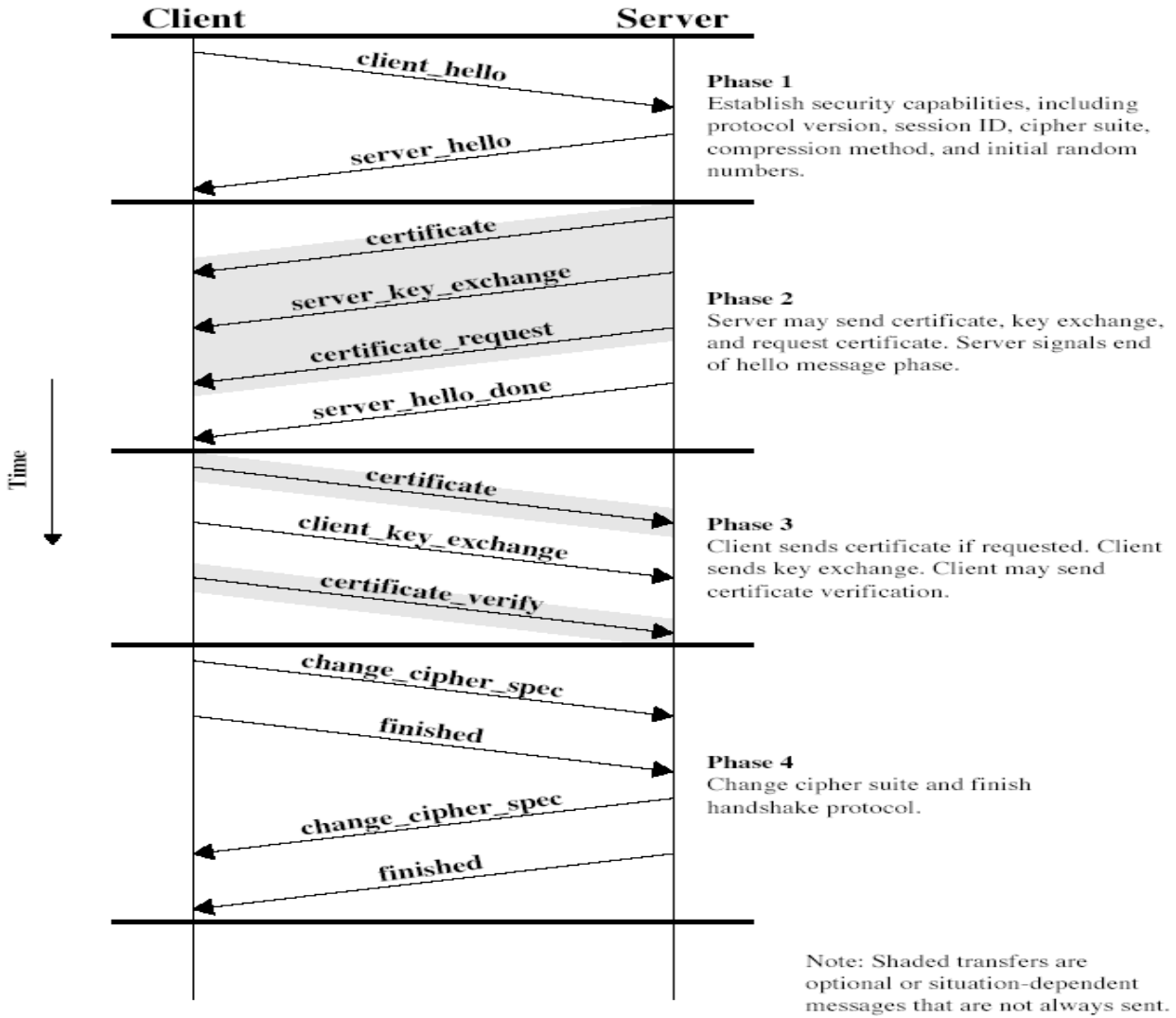
- TLS-RP format
  - **Major Version=3**
  - **Minor Version:**
    - 0 = SSL3.0
    - 1 = TLS1.0
    - 2 = TLS1.1
    - 3 = TLS1.2
    - 4 = TLS1.3
  - **Content-type:**
    - 20 = Change Cihper Spec
    - 21 = Alert
    - 22 = Handshake
    - 23 = Data



TLS Handshake Protocol

- Peer-entity authentication and key exchange is provided by the TLS Handshake phase, performed at the beginning of the communication
- TLS Handshake Protocol allows the server and client to authenticate each other and to negotiate an encryption algorithm and cryptographic keys before the application protocol transmits or receives its first byte of data
  - **the peer's identity can be authenticated using asymmetric, or public key, cryptography (e.g., RSA, DSA, etc.)**
  - **secure negotiation of a shared secret**
    - unavailable to eavesdroppers, or to attackers who can place themselves in the middle of the connection

TLS Handshake Protocol (cont.)



TLS and UDP

- TLS cannot be directly used with UDP, since packets may be lost or reordered (unreliability)
  - **TLS does not allow independent decryption of individual records**
    - the integrity check depends on the sequence number, if record N is not received, then the integrity check on record N+1 will be based on the wrong sequence number and thus will fail
    - the symmetric cipher may also depend on the correct sequence of records
  - **TLS handshake layer assumes that handshake messages are delivered reliably**
    - breaks if those messages are lost

Datagram Transport Layer Security (DTLS)

- Datagram Transport Layer Security (DTLS) Version 1.3
  - **TLS over datagram transport (UDP)**
  - **introduced to provide a security service similar to TLS on top of UDP**
  - **based on TLS**
    - it makes only the minimal changes to TLS required to fix the problems
- Although it is still poorly supported in standard Internet nodes, it is currently the reference security protocol for some IoT protocols and applications
  - **it uses UDP as transport**
  - **It does not suffer from the problems originated by the use of TCP in network-constrained scenarios due to the extremely variable transmission delay and lossy links**

DTLS vs. IPSec

- IPSec and DTLS provide same security features with their own mechanisms. at different stack layers
  - **also the IPSec IKE key agreement reflects almost the same TLS/DTLS handshake function**
- The main advantage of securing communications at transport layer with DTLS consists in allowing applications to directly and easily select which, if any, security service has to be set up
  - **another practical advantage is that the adoption of DTLS can allow for the reuse of the large experience and implementations that came with TLS**
- DTLS has recently received significant attention for constrained networked applications
  - **it has been standardized as the security protocol for CoAP associated to “coaps” URIs**

Hop-by-hop and E2E security

- Providing security at IP layer (through IPSec) or transport layer (through TLS or DTLS) has some advantages
  - **the same standard mechanism and the same implementation can be shared by all applications, resulting in code reuse and reduced code size**
  - **programmers do not have to deal with the implementation of any security mechanism**

Hop-by-hop and E2E security (cont.)

- From the security point of view, one problem of using DTLS or IPSec is that end-to-end (E2E) communication is not guaranteed when intermediate nodes such as proxies or application level gateways are introduced
  - **they provide secure communications hop-by-hop, at IP and transport layers**
  - **in presence of application intermediate nodes, they can assure security only within each hop**
  - **the presence of intermediate nodes may arise for**
    - application reasons
      - caching, data aggregation/processing, application level routing
        - » e.g. IoT gateway, MQTT broker, LoRaWAN App Server (possibly operated by a third party entity)
    - security reasons
      - as security gateway acting as access controller and granting access to the internal network only to trusted nodes
        - » may protect the constrained nodes against flooding or replay attacks carried on by full-powered host attached to the Internet

Hop-by-hop and E2E security (cont.)

- When communication is relayed at application level by intermediate nodes, end-to-end security is guaranteed through transport-level or IP-level mechanisms only if:
  - **communication is secured on each hop**
  - **all intermediate systems are considered trusted**
    - requires additional trust relationships

Application-level security

- A different approach for providing complete end-to-end security is to enforce security directly at application level
  - **simplifies the requirements for underlying layers, and probably reduces the cost, in term of packet size and data processing**
    - since only application data have to be secured and per-data and not per-packet overhead is introduced
  - **multicast communications, and in-network data aggregation is easier to be implemented at application level**
- The main disadvantages are:
  - **the complications introduced for application development**
  - **the overall code size due to poor reuse of software codes**
- These disadvantages are mainly due to the lack of well defined and adopted secure protocols at application level
  - **examples of standards that can be used for this purpose are S/MIME, SRTP, CBOR+COSE**

CBOR

- Concise Binary Object Representation (CBOR)
  - **RFC 8949 (2020, previous RFC in 2013)**
- Data format for binary encoding structured data, designed for
  - **small size**
  - **fast parsing**
  - **extensibility**
- Underlying data model is an extended version of JSON
  - **but compact and machine-efficient**
  - **supports integers, floats, strings, arrays, maps, binary data**
- Deterministic encoding
  - **important for security**
- Alternative of earlier binary serializations
  - such as ASN.1 (with BER and DER), e.g. encoding [1, [2, 3]]
    - ASN.1 BER: 30 0b 02 01 01 30 06 02 01 02 02 01 03
    - CBOR: 82 01 82 02 03

–

Example: JSON vs CBOR

- JSON:

```
{"temp":22,"humidity":48,"battery":2980}
```

  - size= 40 B

- CBOR:

```
A3                                  # map(3)
   64                               # text(4)
      74 65 6D 70                   # "temp"
   16                               # unsigned(22)
   68                               # text(8)
      68 75 6D 69 64 69 74 79       # "humidity"
   18 30                            # unsigned(48)
   67                               # text(7)
      62 61 74 74 65 72 79          # "battery"
   19 0B A4                         # unsigned(2980)
```

  - size= 29 B

- CBOR with numeric keys: `{1:22, 2:48, 3:2980}`

```
A3
01 16       # 1 → 22
02 18 30    # 2 → 48
03 19 0B A4 # 3 → 2980
```

  - size= 10 B

CBOR encoding

- Every CBOR data item starts with an initial byte:
  - **3 bits: Major Type**
    - defines what kind of data follows
      - 0 Unsigned integer
      - 1 Negative integer
      - 2 Byte string (binary data)
      - 3 Text string (UTF-8)
      - 4 Array
      - 5 Map (key-value pairs)
      - 6 Tag (semantic annotation)
      - 7 Floating point / simple values
  - **5 bits: Additional Information**
    - indicates either:
      - the value itself (if small), or
      - how many extra bytes follow

CBOR encoding (cont.)

- CBOR encodes each piece of data as a byte sequence where the first byte tells you the type and size
- Encoding process:
  1) **take the data structure (e.g., JSON)**
  2) **for each element:**
     - write type + length/value
     - append extra bytes if needed
  3) **concatenate everything into a byte stream**

CBOR in IoT Protocols

- Used in:
  - **CoAP (Constrained Application Protocol)**
    - Object Security for Constrained RESTful Environments (OSCORE)
  - **LwM2M (Lightweight M2M)**
    - from the Open Mobile Alliance
  - **Matter (smart home)**
    - from Connectivity Standards Alliance

- Often combined with:
  - **COSE (CBOR Object Signing and Encryption)**

COSE

- CBOR Object Signing and Encryption
  - **RFC 8949 (2020)**
  - **specified how to create and process secure messages using CBOR**
    - analogous to JOSE (Javascript Object Signing and Encryption) with JWT, JWS, JWE) but binary
- Supports:
  - **encryption**
  - **digital signatures**
  - **message authentication (MAC)**

COSE messages

- Most COSE messages follow this pattern:

  [protected headers,
  unprotected headers,
  payload / ciphertext,
  signature / tag]

  - **where:**
    - protected header: CBOR map, integrity protected
      - Typically contains:
        - » algorithm (e.g., AES-GCM, EdDSA)
        - » key ID
    - unprotected headers: CBOR map, used for non-critical metadata
    - payload: plaintext or encrypted
    - signature / tag: ensures authenticity (authentication and integrity)
        - » signature → digital signature
        - » encryption → AEAD tag (e.g., GCM)

Main COSE message types

- COSE_Encrypt / COSE_Encrypt0
  - **for encryption with a AEAD algorithm (e.g., AES-GCM, AES-CCM)**
  - **structure**

```
COSE_Encrypt0 = [
  protected,
  unprotected,
  ciphertext
]
```

- COSE_Sign / COSE_Sign1
  - **for digital signature**
  - **structure**

```
COSE_Sign1 = [
  protected,
  unprotected,
  payload,
  signature
]
```

- COSE_Mac / COSE_Mac0
  - **for MAC (shared key)**
  - **similar to encryption, but no confidentiality**

OSCORE

- Object Security for Constrained RESTful Environments (OSCORE)
  - **RFC 8613 (2019)**
- Method for application-layer protection of CoAP
  - **provides end-to-end protection between endpoints**
    - designed for constrained nodes and networks supporting a range of proxy operations
    - it uses CBOR+COSE

OSCORE (cont.)

- OSCORE transforms a CoAP message

  [CoAP headers] [CoAP options] [Payload]

  - **into**

    [Outer message] [OSCORE option] [Encrypted inner message]

  - **where**
    - Outer message (visible, partially authenticated):
      - dummy CoAP code (for keep CoAP semantic, POST/Changed)
      - minimal routing info
        - » message ID, token, some options (e.g., Proxy-Uri)
    - Inner message (encrypted + authenticated):
      - CoAP code (GET, POST, etc.)
      - sensitive options (e.g. Uri-Path)
      - payload

Key Agreement and Distribution

- Key distribution and management is a crucial issue that needs to be addressed when security mechanisms have to be adopted
- Asymmetric (public-key) cryptographic algorithms are often the basis for key agreement protocols
  - e.g. Diffie–Hellman
- In IoT environments particular attention must be paid, due to processing constrains
  - **type of used algorithm**
  - **frequency of key exchanges**
- Alternative approaches may involve
  - **key pre-distribution**
  - **use of a third party Key Distribution Center (KDC)**

Security bootstrapping

- All key agreement protocols require that some credentials have been previously installed and configured on nodes, so that the key agreement procedure can occur securely
- Bootstrapping refers to the processing operations required before the network can operate
  - **it is a very important phase in the lifecycle of smart objects and can affect the way they behave in operational conditions**
  - **in smart objects it can be complicated by the absence of UIs**
  - **in this case, the bootstrapping may exploit:**
    - hardware pre-configured settings
    - physical environment where the node is bootstrapping or operates

Lightweight Cryptography

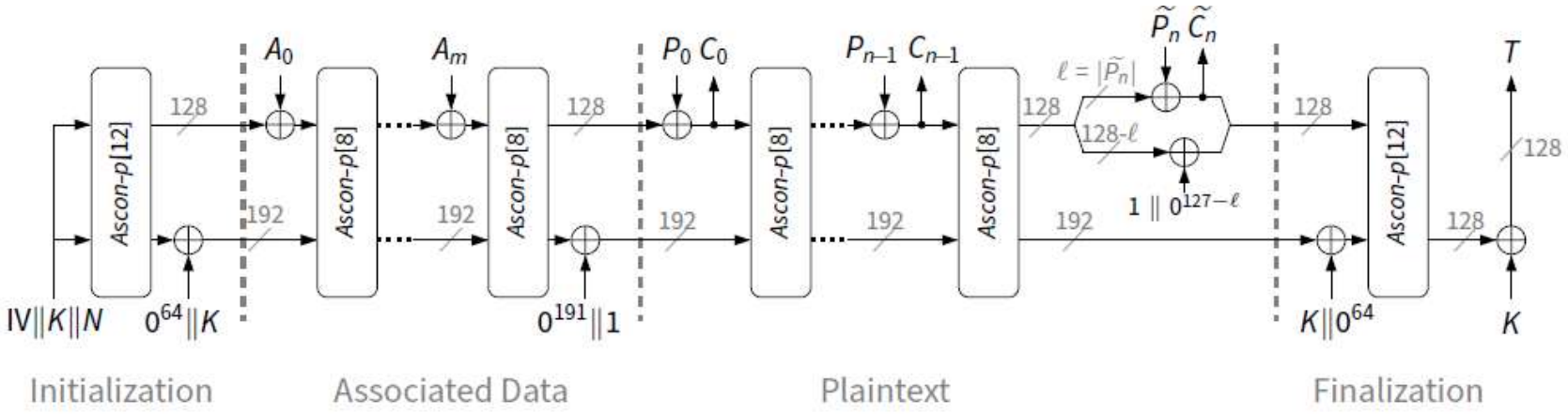
- Smart objects may feature constrained resources, such as low computation capabilities, little memory, and limited battery
- Security mechanism/algorithms must necessarily take into account all these limitations
- Lightweight Cryptography (LWC) aims at the design of ciphers that might meet the requirements set by the use of smart objects
  - **cryptography tailored to (extremely) constrained devices**
  - **algorithms with smaller footprint, low energy consumption, and low computational power needs**
- “Lightweight" should not be mistaken with weak (in terms of cryptographic protection)
  - **aim at providing sufficient security in the environment of restricted resources as can be found in many ubiquitous devices**
  - **trade-offs between security level, cost, and performance**
- Not intended for all-powerful adversaries

NIST lightweight cryptography standard

- NIST published a call for cryptography algorithms for constrained environment, in 2018
  - **received 57 submissions**
  - **10 finalists**
  - **in February 2023 the selection of the Ascon algorithm family**
  - **in November 2024 released the initial public draft**
- Family of symmetric-key cryptographic primitives
  - **Authenticated Encryption with Associated Data (AEAD): Ascon-AEAD128**
  - **Hash: Ascon-Hash256**
  - **Extendable Output Function (XOF): Ascon-XOF128, and Ascon-CXOF128**
- Characterized by lightweight permutation-based primitives
- Viable alternative when the Advanced Encryption Standard (AES) may not perform optimally

Ascon-AEAD128

- Ascon-AEAD128 structure
  - **comprises four phases: initialization, associated data processing, plaintext processing, and finalization**



Energy consumption of asymmetric algorithms

- Energy cost of digital signature algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Sign (mJ)	Verify (mJ)
RSA	1,024	270.13	546.50	15.97
DSA	1,024	293.20	313.60	338.02
ECDSA	163	226.65	134.20	196.23
ECDSA	193	281.65	166.75	243.84
ECDSA	233	323.30	191.37	279.82
ECDSA	283	504.96	298.86	437.00
ECDSA	409	1034.92	611.40	895.98

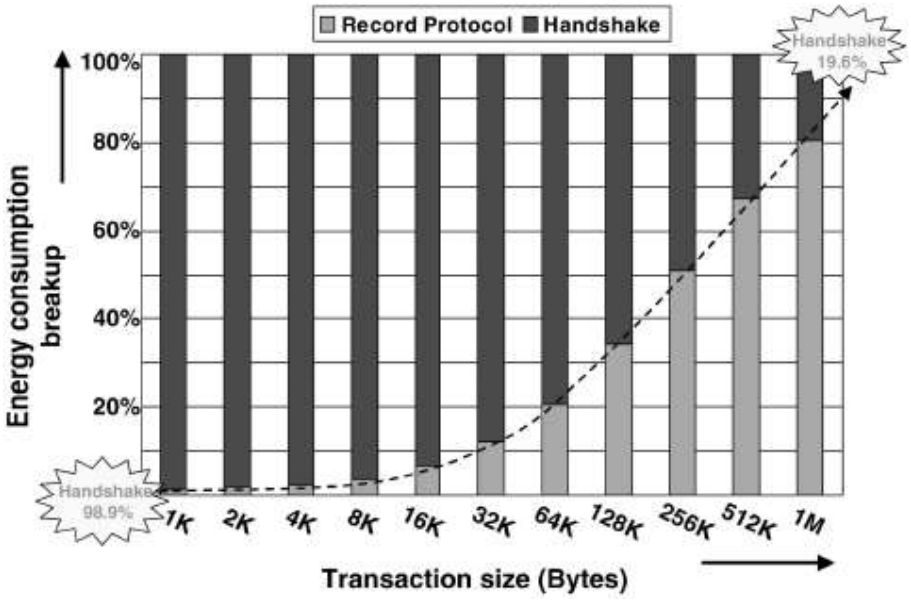
- Energy cost of key exchange algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Key exchange (mJ)
DH	1,024	875.96	1,046.5
ECDH	163	276.70	163.5
DH	512	202.56	159.6

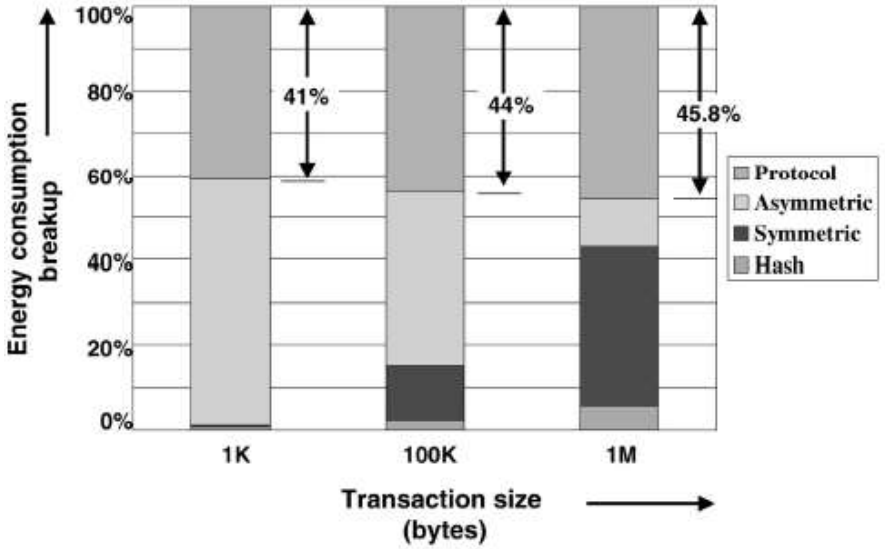
[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

TLS energy consumption (cont.)

- Handshake vs. RP energy consumption:



- TLS energy consumption into cryptographic and non-cryptographic components:



[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006